

# LangChain Agents & Tools



**SoftUni Team**  
**Technical Trainers**



**SoftUni**



**Software University**

<https://about.softuni.bg>

# Table of Contents

1. LangChain Introduction
2. Models & Messages
3. Tools
4. Agents
5. Middlewares





**LangChain**

**LangChain**

Understanding LangChain and its Core Features

# What is LangChain?

- **Abstraction** over AI providers
- **Python** (preferred) and TypeScript integration
- Build **Agents** and Automations in the same environment

```
pip install -U langchain  
# Requires Python 3.10+
```

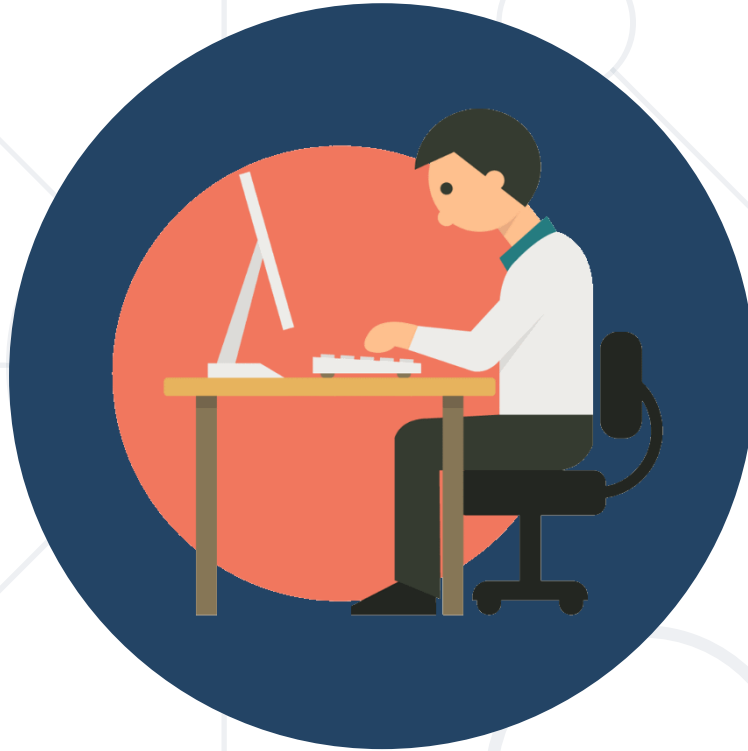
```
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage  
from langchain_openai import ChatOpenAI  
  
chat_model = ChatOpenAI(model="gpt-4o-mini", temperature=0.5)  
  
conversation = [  
    SystemMessage(content="You are a sarcastic AI assistant for programmers. Keep it short."),  
    HumanMessage(content="Why isn't my code compiling?")  
]  
  
response = chat_model.invoke(conversation)
```

# Why not use APIs?

- Integrate any **LLM** provider
- Retrieve any **information**
- Search any **engine**
- Load any **document**
- Cons: needs to catch up with providers

# Why Trust LangChain?

- **Open Source**
- Wide **Adoption**
- Active **Community**
- **Ease** of Integration
- Documentation and Support



# Practice

Exercise – Try LangChain Quickstart



# Models & Messages

The Stateless Protocol

# The LLM as a Function

- **Stateless** Nature
  - LLMs do not "remember" previous calls.
  - Every request must contain the full history.
- **Unified** API
  - Same `invoke()` method for OpenAI, Anthropic, or local models (Llama).
- Configurable **Parameters**

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0.8,
    max_tokens=100
)
```

- Evolution of I/O
  - Shift from raw strings to structured BaseMessage objects.
- Message Hierarchy
  - **SystemMessage**: Sets the behavior/persona.
  - **HumanMessage**: User-provided input.
  - **AIMessage**: Model-generated response.
  - **ToolMessage**: Result of an external function execution.

```
from langchain_core.messages import SystemMessage, HumanMessage

messages = [
    SystemMessage(content="You only reply in uppercase letters."),
    HumanMessage(content="How are you?")
]

response = llm.invoke(messages)
```

- Role Definition
  - Defines constraints (e.g., "Answer only using the provided context").
- Output Formatting
  - Forces the model to respond in JSON, XML, or specific schemas.
- Context Injection
  - Where RAG data is usually placed.

```
from langchain_openai import ChatOpenAI

json_llm = ChatOpenAI(model_kwargs={"response_format": {"type": "json_object"}})

messages = [
    SystemMessage(content="Return a JSON with a 'status' key."),
    HumanMessage(content="System check.")
]
```

- **Windowing**

- Keeping only the last  $N$  messages to save tokens.

- **Summarization**

- Compressing old history into a single summary message.

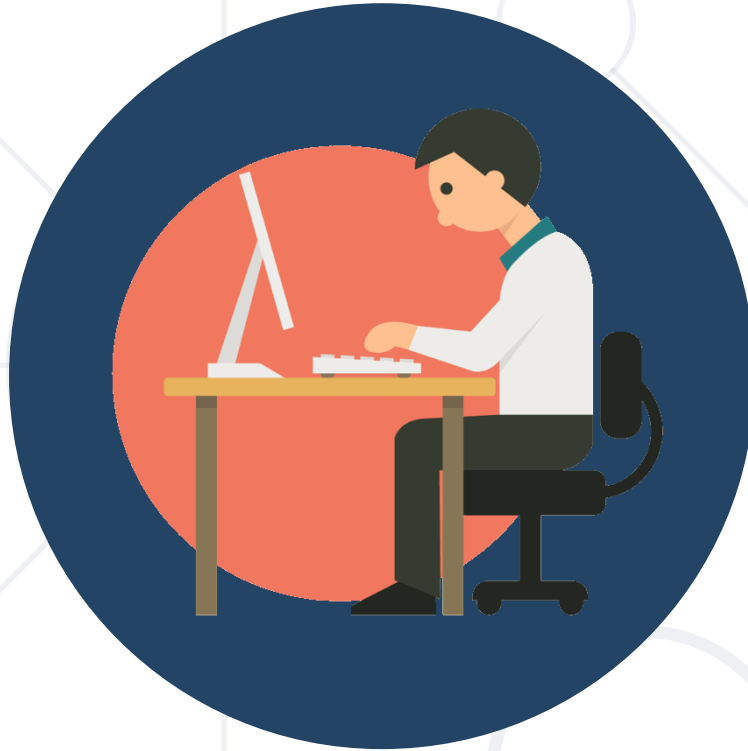
- **Thread Isolation**

- Managing separate message lists for different users.

```
from langchain_core.messages import AIMessage, HumanMessage

full_history = [
    HumanMessage(content="Hi"),
    AIMessage(content="Hello"),
    HumanMessage(content="What is 2+2?"),
    AIMessage(content="4"),
    HumanMessage(content="And 3+3?")
]

windowed_history = full_history[-3:]
response = llm.invoke(windowed_history)
```



# Practice

Exercise – Experiment with Models and Messages



# Tools

Giving the Model Hands and Eyes

- Executable Python **Functions**
  - Any function can be a tool using the *@tool* decorator.
- Self-Documenting Code
  - The model uses your **Docstrings** to understand when to use the tool.
  - The model uses Type Hints to understand how to format arguments.
- JSON Schema Generation
  - LangChain automatically creates the schema required by the Model API.

```
from langchain_core.tools import tool

@tool
def get_database_status(cluster_name: str) -> str:
    """Returns the current health status of a database cluster."""
    return "healthy"
```

- The "Attach" Step
  - Tools are bound to the model via `.bind_tools([my_tool])`.

```
llm_with_tools = llm.bind_tools([get_database_status])  
  
response = llm_with_tools.invoke("Is the US-EAST cluster healthy?")
```

- API Awareness
  - This informs the LLM of the available "capabilities" in the current context.
- Forced Tool Calling
  - You can force the model to always use a specific tool.

- Detection
  - Model detects it cannot answer the question alone.
- Invocation Request
  - Model returns a tool\_calls payload (function name + arguments).
- Local Execution
  - Crucial: The Python code runs on your server, not on OpenAI/Anthropic servers.
- Feedback Loop
  - The result is sent back to the model for a final summary.

```
tool_call = response.tool_calls[0]
function_name = tool_call["name"]
arguments = tool_call["args"]

execution_result = get_database_status.invoke(arguments)
```

- Retrieve information from different **sources** (50+ and counting)
- Examples: Google Drive, arXiv, ElasticSearch
- Input: string query
- Output: Document class - **LINK**

```
from langchain_community.retrievers import ArxivRetriever

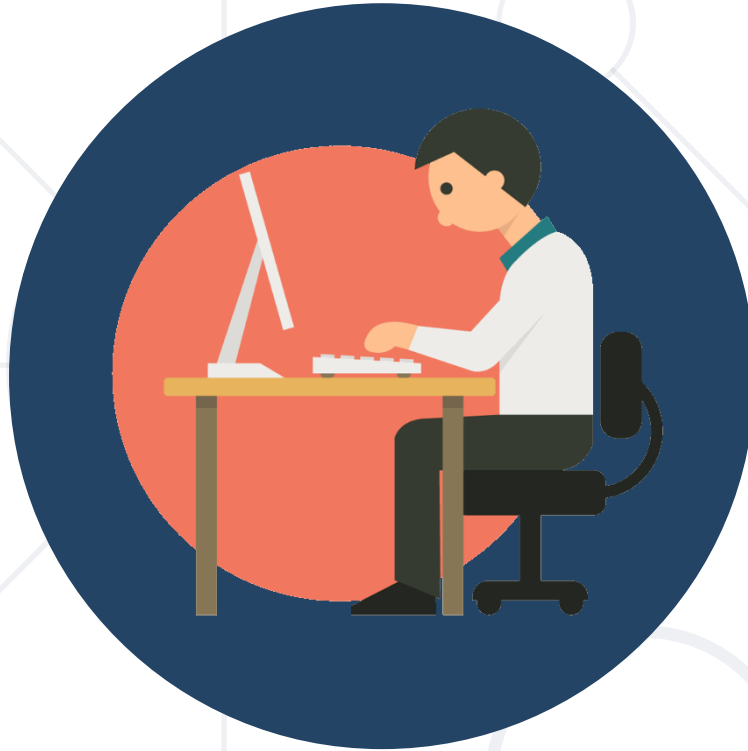
retriever = ArxivRetriever(load_max_docs=2, get_full_documents=True)

search_query = "machine learning"

docs = retriever.invoke(search_query)

if docs:
    print(f"Found docs: {len(docs)}")
    print(f"Header of the first document: {docs[0].metadata['Title']}")
else:
    print("No documents found.")
```

- Ingests and processes **documents**
- Supports structured/unstructured **formats**
- Web pages, **PDFs**, Cloud Storage, **Figma** files and much more
- Foundation for **RAG**: Loaders feed data into a **Vector Store** (which acts as a Retriever)
- **Loaders vs. External Retrievers**: Not all retrievers need a vector store (e.g., **Wikipedia**, **Arxiv**, **Tavily** retrieve data dynamically)



# Practice

Exercise – Define and Bind Tools



# Agents

Orchestrating Reasoning and Action (ReAct)

# Model vs. Agent

- Model = Engine
  - Stateless, calculates next tokens, decides intent.
- Agent = Runtime
  - Stateful, runs the loop, executes the code.
- **Autonomy**
  - Agents decide how many steps are needed to finish a task.

```
messages = [HumanMessage(content="Check DB status and restart if failed.")]

while True:
    response = llm_with_tools.invoke(messages)
    messages.append(response)
    if response.tool_calls:
        for tool_call in response.tool_calls:
            result = get_database_status.invoke(tool_call["args"])
            messages.append(ToolMessage(
                content=str(result),
                tool_call_id=tool_call["id"]
            ))
            continue
        break
```

- Reasoning (Thought)
  - Model writes down what it plans to do.
- Acting (Action)
  - Model selects a tool and provides arguments.
- Observation
  - The outcome of the tool is observed by the agent.
- Reflection
  - Model analyzes the observation and decides if it is done.

- **Termination** Logic
  - The loop continues as long as the model returns tool\_calls.
- The **Stop Signal**
  - The loop breaks when the model returns a standard text response.
- Safety Latency
  - Developers set max\_iterations to prevent infinite loops.

```
from langchain_core.messages import ToolMessage

if response.tool_calls:
    for tc in response.tool_calls:
        result = get_database_status.invoke(tc["args"])

        msg = ToolMessage(content=str(result), tool_call_id=tc["id"])
        messages.append(msg)
```

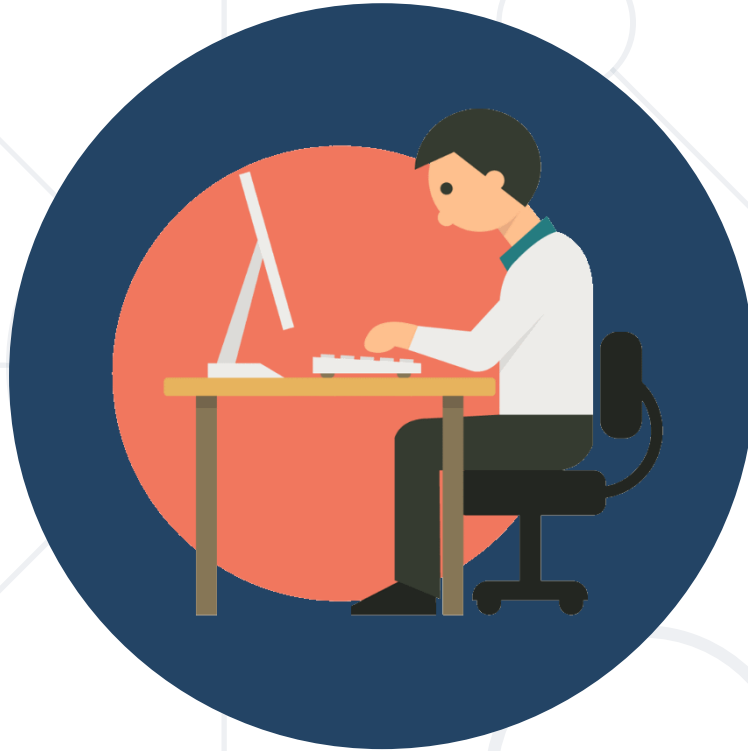
- **Autonomous** Orchestrator: Automates the Reasoning-Action cycle without manual while loops.
- **Simplified** Config: Initializes complex agents using only a model, tool list, and prompt.
- **Standardized** I/O: Handles message history and tool result injection automatically.
- **Provider Agnostic**: Works across different LLM providers using a unified interface.

```
from langchain.agents import create_agent

agent = create_agent(
    model="openai:gpt-4o",
    tools=[manage_server],
    system_prompt="You are a server administrator."
)

result = agent.invoke({
    "messages": [("user", "Restart srv-99")]
})

print(result["messages"][-1].content)
```



# Practice

Exercise – Create your first LangChain Agent



# Middlewares

Intercepting and Controlling the Flow

# Why Middleware?

- Separation of Concerns
  - Keep core logic clean while adding "cross-cutting" features.
- Observability
  - Log inputs/outputs for auditing or debugging.
- Security
  - Sanitize user input or validate tool outputs before the model sees them.

```
from langchain_core.runnables import RunnableLambda

def audit_logger(state: dict) -> dict:
    print(f"AUDIT: Received input {state}")
    return state

middleware = RunnableLambda(audit_logger)
```

- Composition
  - Using the `|` operator to pipe data through different steps.
- Transformation
  - `RunnableLambda`: Custom functions to modify data in-flight.
- Parallelization
  - `RunnableParallel`: Run multiple chains/models at once.

```
from langchain_core.runnables import RunnableParallel

data_pipeline = RunnableParallel(
    uppercase=lambda x: x["text"].upper(),
    word_count=lambda x: len(x["text"].split())
)

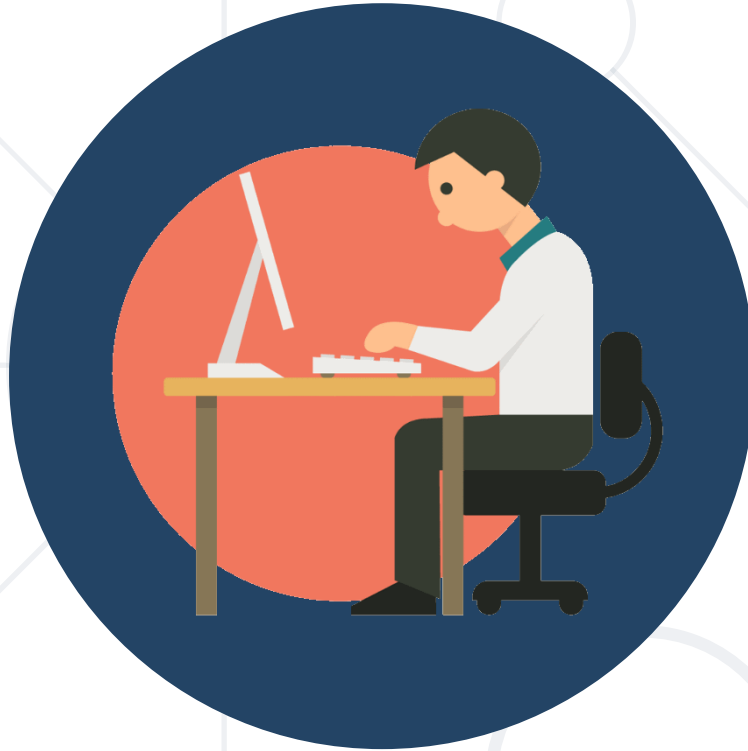
result = data_pipeline.invoke({"text": "Hello world"})
```

- Cost Tracking
  - Calculate token usage after every model call.
- Caching
  - Return cached results for repeated expensive queries.
- Input Mutation
  - Uppercasing, translating, or enriching data before it hits the prompt.
- Guardrails
  - Block responses that contain sensitive data (PII).

```
from langchain_openai import ChatOpenAI
from langchain_anthropic import ChatAnthropic

primary_model = ChatOpenAI(model="gpt-4o")
fallback_model = ChatAnthropic(model="claude-3-haiku-20240307")

resilient_pipeline = primary_model.with_fallbacks([fallback_model])
```



# Practice

Exercise – Create your first LangChain Middleware

- **Models vs. Agents:** The Model is a stateless engine (**Brain**); the Agent is a stateful loop (**Orchestrator**) that executes actions.
- **The ReAct Pattern:** Agents cycle through **Reasoning** (Thought) and **Acting** (Tool Call) until the goal is achieved.
- **Tools as Plugins:** Functions, Retrievers, and Loaders act as the model's interface to external data and APIs.
- **Agentic Control:** Logic is governed by **Middlewares** and **Graphs**, ensuring the agent stays within defined safety and business guardrails.

